

FirstKnock 300 sq mile Chunking Validation Spec

FirstKnock 300 sq mile Chunking Validation Spec

Engineering Specification: Geographic Chunking, Deduplication, Job State Tracking & Coverage Verification

March 31, 2026

This specification defines the chunking strategy, deduplication logic, job state tracking, error handling, and coverage verification approach for fetching property data across a 300 square mile target area using the RentCast API. The goal is to ensure complete, deduplicated, auditable coverage of any 300 sq mile geographic region used by FirstKnock sales representatives.

300 sq mi Target Coverage Area	16 Sub-circles (r=5 config)	~90.7% Hexagonal Packing Efficiency	5 miles Recommended Sub-circle Radius
---------------------------------------	------------------------------------	--	--

Table of Contents

- 1. Overview & Purpose
- 2. Sub-circle Radius and Grid Math
- 3. Coverage Area Geometry
- 4. Recommended Configuration: 16 Sub-circles at r=5 miles
- 5. Alternative Configuration: 9 Sub-circles at r=6 miles
- 6. RentCast API Radius Parameter
- 7. Overlap Handling and Deduplication
- 8. Why Overlap Is Intentional
- 9. Deduplication Strategy
- 10. Dedup Metrics
- 11. Job State Tracking Requirements
- 12. Required Job Metrics
- 13. Persistent Storage (job_runs schema)
- 14. Invariant Check
- 15. Error Handling Per Sub-circle
- 16. Core Principle
- 17. Error Response Matrix
- 18. Failed Chunk Log Schema
- 19. Post-Job Retry
- 20. Coverage Verification Approach
- 21. Bounding Box Verification
- 22. Grid Cell Density Check
- 23. Coverage Verification Log Schema
- 24. Validation Checklist for processFetchChunk
- 25. Implementation Notes from Source Documents

1. Overview & Purpose

This specification defines the chunking strategy, deduplication logic, job state tracking, error handling, and coverage verification approach for fetching property data across a 300 square mile target area using the RentCast API. The goal is to ensure complete, deduplicated, auditable coverage of any 300 sq mile geographic region used by FirstKnock sales representatives.

The RentCast API enforces a maximum radius of 100 miles per call, but practical performance and result-count constraints require sub-dividing any large geographic target area into smaller overlapping sub-circles ("chunks"). This specification prescribes the exact grid geometry, overlap handling, deduplication algorithm, job state persistence, error recovery, and post-job coverage verification required to implement a production-grade chunked fetch pipeline.

SCOPE: This spec governs the processFetchChunk function and all supporting infrastructure. It does not cover BatchData validation logic, heuristic scoring, or the Phase 1 /properties deed-record pipeline — those are addressed in the RentCast Deep Research Report and the Reduce RentCast False Alarms document.

2. Sub-circle Radius and Grid Math

2.1 Coverage Area Geometry

- Total target coverage area: 300 sq miles
- A single circle covering 300 sq miles would require radius $r = \sqrt{300/\pi} \approx 9.77$ miles — but this assumes a perfect circle target area, which is unrealistic for geographic coverage of a sales territory.
- For a square-ish 300 sq mile area (~17.3 times 17.3 miles), a hexagonal grid of sub-circles provides optimal coverage efficiency.

2.2 Recommended Configuration: 16 Sub-circles at r=5 miles

Table 1 — Recommended 16-Circle r=5 Configuration Parameters

Parameter	Value
Sub-circle radius	5 miles
Area per circle (theoretical)	$\pi \times 5^2$ approx 78.54 sq mi
Grid layout	4-column times 4-row offset (hexagonal)
Total sub-circles	16
Hexagonal horizontal spacing	$2r \times \cos(30^\circ)$ approx 8.66 miles
Hexagonal vertical spacing	$1.5r = 7.5$ miles

Theoretical total coverage	16 times 78.54 approx 1,257 sq mi
Hexagonal packing efficiency	~90.7%
Effective coverage (after overlap)	approx 300 sq mi with ~10% boundary overlap

2.3 Alternative Configuration: 9 Sub-circles at r=6 miles

Table 2 — Alternative 9-Circle r=6 Configuration Parameters

Parameter	Value
Sub-circle radius	6 miles
Area per circle	pi times 6 ² approx 113.1 sq mi
Grid layout	3times3 uniform grid
Total sub-circles	9
Overlap	~20% (simpler math, more overlap)

RECOMMENDATION: Use the 16-circle r=5 configuration for tighter coverage control and lower per-call result counts. Use the 9-circle r=6 configuration for simpler implementation when overlap is acceptable.

2.4 RentCast API Radius Parameter

- The RentCast API accepts radius in miles as a query parameter.
- Based on research documentation, no hard cap is explicitly documented, but practical limits suggest staying under 25 miles per call to avoid excessive result counts requiring heavy pagination.
- At r=5, expect approximately 50–200 properties per sub-circle depending on density.
- At r=6, expect approximately 100–400 properties per sub-circle in suburban areas.

3. Overlap Handling and Deduplication

3.1 Why Overlap Is Intentional

Sub-circle boundaries WILL overlap — this is by design to prevent coverage gaps at grid boundaries. Every property in an overlap zone will appear in 2 or more chunk results. This is expected and handled in the merge phase.

3.2 Deduplication Strategy

CRITICAL: Do NOT deduplicate within a single chunk. Deduplication MUST occur after all chunks

complete, during the merge phase.

Primary dedup key: id — RentCast native property identifier. Format: 'Street-Address-City-State-Zip' (e.g., '123-Main-St-Austin-TX-78701')

Secondary dedup key: formattedAddress — normalized address string (fallback if id is missing or malformed)

Deduplication algorithm:

1. Build a Map<string, PropertyRecord> keyed by property id.
2. For each record in the merged result set:
3. If id not in Map → insert record
4. If id already in Map → compare lastSaleDate or lastSeenDate
5. Keep the record with the MORE RECENT date
6. Discard the older duplicate
7. Collision resolution: Always prefer the more recently dated record to ensure data freshness.

3.3 Dedup Metrics

After deduplication, compute the following metrics and log them to the job_runs table:

- duplicate_count = total_fetched - total_unique
- If duplicate_count === 0 and total_chunks > 1, flag as a potential dedup failure — overlapping grids should always produce some duplicates.

4. Job State Tracking Requirements

All metrics must aggregate across the FULL multi-chunk job, not just the last chunk.

4.1 Required Job Metrics

Table 3 — Required Job Metrics for job_runs Table

Metric	Description
total_fetched	Sum of all records returned across all sub-circle API calls (before dedup)
total_unique	Count of unique records after cross-chunk deduplication
total_inserted	Count of records successfully written to database
total_api_calls	Count of HTTP requests made (including pagination calls per sub-circle)
total_chunks	Total number of sub-circles in the grid
completed_chunks	Sub-circles that completed successfully

failed_chunks	Sub-circles that failed (with coordinates and error logged)
duplicate_count	total_fetched - total_unique (overlap records removed)
job_start_time	ISO timestamp when job began
job_end_time	ISO timestamp when job completed
job_duration_ms	job_end_time - job_start_time in milliseconds

4.2 Persistent Storage — job_runs SQL Schema

Store all job state in a job_runs table for a full audit trail. The following SQL schema must be implemented exactly as specified:

Table 4 — job_runs Table Schema

Column	Type	Notes
id	UUID PRIMARY KEY	DEFAULTgen_random_uuid()
job_id	TEXT NOT NULL	Unique identifier for the job run
area_id	TEXT	Identifier for the target geographic area
total_chunks	INTEGER	Total sub-circles in the grid
completed_chunks	INTEGER	Successfully completed sub-circles
failed_chunks	INTEGER	Failed sub-circles (logged separately)
total_fetched	INTEGER	Raw records before dedup
total_unique	INTEGER	Records after cross-chunk dedup
total_inserted	INTEGER	Records written to database
total_api_calls	INTEGER	Total HTTP requests including pagination
duplicate_count	INTEGER	total_fetched - total_unique
job_start_time	TIMESTAMPTZ	ISO timestamp — job start
job_end_time	TIMESTAMPTZ	ISO timestamp — job end
job_duration_ms	BIGINT	Duration in milliseconds

coverage_verification	JSONB	Bounding box and grid cell check results
failed_chunk_log	JSONB	Array of failed chunk records with coordinates and errors
created_at	TIMESTAMPTZ	DEFAULTNOW()

4.3 Invariant Check

After job completion, assert all three of the following invariants. Any violation indicates a logic error and must halt further processing pending investigation:

- 1. total_fetched neq total_inserted — if equal and total_chunks > 1, deduplication is likely broken
- 2. total_unique leq total_fetched — always true; if violated, logic error in dedup algorithm
- 3. completed_chunks + failed_chunks = total_chunks — all chunks must be accounted for

5. Error Handling Per Sub-circle

5.1 Core Principle

NEVER abort the entire job due to a single sub-circle failure. Each sub-circle fetch is wrapped in an independent try/catch. Failed chunks are logged, skipped, and retried after the main job completes.

5.2 Error Response Matrix

Table 5 — Error Response Matrix for Sub-circle Fetch Operations

HTTP Status	Action
429 Rate Limit	Exponential backoff: wait 5s → retry; wait 10s → retry; wait 20s → retry; if still failing, mark chunk as failed and continue to next chunk
500 Server Error	Wait 60s, retry once; if still fails, mark chunk as failed and continue to next chunk
401 Unauthorized	HALT entire job immediately — API key issue requires human intervention. Do not retry under any circumstances.
Network Timeout	Retry up to 3 times with 5s delay between attempts; if all fail, mark chunk as failed

Other 4xx	Log error with full request details, mark chunk as failed, continue to next chunk
-----------	---

5.3 Failed Chunk Log Schema

Each failed chunk must be logged with the following JSON structure and stored in the failed_chunk_log JSONB column of the job_runs table:

Table 6 — Failed Chunk Log JSON Schema Fields

Field	Type	Description	Example
chunk_id	string	Unique identifier for the sub-circle chunk	"chunk_07"
center_lat	number	Latitude of the sub-circle center point	30.2672
center_lon	number	Longitude of the sub-circle center point	-97.7431
radius	number	Radius in miles used for this chunk	5
error_code	integer	HTTP status code returned	429
error_message	string	Human-readable error description	"Too Many Requests"
timestamp	ISO 8601	UTC timestamp when the error occurred	"2025-04-07T14:23:11Z"
retry_count	integer	Number of retry attempts made	3
retry_exhausted	boolean	True if all retries were consumed	true

5.4 Post-Job Retry

After the main job completes, if failed_chunks > 0, execute the following retry sequence:

1. Extract all failed chunk records from the failed_chunk_log JSONB column
2. Trigger a targeted re-run using only the failed chunk coordinates
3. Merge re-run results into the existing job's result set
4. Re-run deduplication across the full merged set (including newly recovered chunks)
5. Update the job_runs record with final metrics reflecting the completed retry pass

6.1 Bounding Box Verification

After all chunks complete and deduplication is done, execute the following bounding box verification sequence:

1. Compute the bounding box of all returned property coordinates: min_lat, max_lat, min_lon, max_lon
2. Compare against the expected bounding box of the 300 sq mile target area
3. If actual bounding box coverage < 95% of expected area rightarrow flag for manual review and log to coverage_verification JSONB column

6.2 Grid Cell Density Check (Optional but Recommended)

1. Divide the target area into a 10times10 grid of 100 cells (each ~0.3 sq mi)
2. For each cell, count properties with coordinates falling within that cell
3. Flag any cell with 0 properties as a potential coverage gap
4. Log: empty_cells: [{ row: 3, col: 7, bbox: {...} }, ...]

6.3 Coverage Verification Log Schema

The coverage_verification JSONB column in job_runs must be populated with the following structure after each job completes:

Table 7 — Coverage Verification JSONB Schema Fields

Field	Type	Description
expected_bbox.min_lat	number	Minimum latitude of the expected target bounding box
expected_bbox.max_lat	number	Maximum latitude of the expected target bounding box
expected_bbox.min_lon	number	Minimum longitude of the expected target bounding box
expected_bbox.max_lon	number	Maximum longitude of the expected target bounding box
actual_bbox.min_lat	number	Minimum latitude of all returned property coordinates
actual_bbox.max_lat	number	Maximum latitude of all returned property coordinates

actual_bbox.min_lon	number	Minimum longitude of all returned property coordinates
actual_bbox.max_lon	number	Maximum longitude of all returned property coordinates
coverage_pct	number	Percentage of expected bounding box covered by actual results (e.g., 97.2)
empty_cells	array	Array of grid cells with 0 properties — each entry contains row, col, and bbox
flagged_for_review	boolean	True if coverage_pct < 95% or any critical empty cells detected

7. Validation Checklist for processFetchChunk

Use this checklist before deploying any chunking implementation. All items must be verified. Items marked HIGH must pass before any production deployment. Items marked MEDIUM should pass before any production deployment but may be deferred to a follow-up sprint with documented justification.

Table 8 — processFetchChunk Pre-Deployment Validation Checklist

#	Checklist Item	Priority	Status
1	Sub-circle radius math produces correct coverage for target area size (r=5 for 16-circle grid, r=6 for 9-circle grid)	HIGH	[]
2	Grid spacing accounts for overlap: horizontal spacing approx 8.66 mi (< 2r=10), vertical spacing approx 7.5 mi (< 2r=10) — gaps are	HIGH	[]
3	Cross-chunk deduplication runs AFTER all chunks complete (not within individual chunks)	HIGH	[]

4	Job metrics aggregate across ALL chunks (not just last chunk — use running totals)	HIGH	[]
5	Each sub-circle fetch is wrapped in an independent try/catch block	HIGH	[]
6	Failed chunks are logged with full coordinates, error code, and timestamp	HIGH	[]
7	Failed chunks are retried in a separate pass after main job completes	HIGH	[]
8	HTTP 401 halts entire job immediately with alert; all other errors continue to next chunk	HIGH	[]
9	Coverage verification (bounding box + optional grid cell check) runs after merge phase	MEDIUM	[]
10	total_fetched neq total_inserted assertion runs post-job (if equal with multi-chunk job, dedup is broken)	HIGH	[]
11	job_runs table updated with final metrics and coverage_verification result	HIGH	[]
12	Duplicate count logged: duplicate_count = total_fetched - total_unique	HIGH	[]

8. Implementation Notes from Source Documents

The connected source documents — the RentCast API Deep Research Report (Blocks 1 & 2), the

8.1 RentCast API Pagination Behavior

- Maximum records per response: 500 (applies to both /properties and /listings/sale endpoints)
- Pagination is controlled via limit (max 500) and offset (starting index) parameters
- Termination condition: stop paginating when the number of objects returned is strictly less than the limit value, or when the API returns an empty array []
- For Las Cruces with saleDateRange=180, each individual ZIP code is expected to return fewer than 500 records — a single API call per ZIP retrieves the complete dataset without pagination
- At r=5 sub-circle radius in suburban areas, expect 50–200 properties per chunk — pagination is unlikely but must be implemented defensively

8.2 BatchData Record Counts Per Area

Table 9 — BatchData Record Counts and Costs by Validation Strategy

Scenario	Records Sent to BatchData	Monthly Cost (PAYG @ \$0.10)	Notes
Full validation (no filtering)	~215 records/area	\$21.50/area	Current state — unsustainable at scale
Optimized (Freshness Window + Cross-Reference)	~50 records/area	\$5.00/area	Recommended for 1-rep deployments
Delta validation only (weekly)	~13 records/week	\$1.30/week	Highest ROI — requires persistent listingId store
Confidence scoring pre-filter	~118 records/area	\$11.80/area	Eliminates obvious expirations (DOM >90 days)

8.3 False Alarm Reduction — The Lag Window Rule

The Reduce RentCast False Alarms document prescribes the following rule for Phase 2 (listings_records) ingestion, which directly affects how many records from the chunking pipeline are forwarded to BatchData validation:

- New Rule: If a property has a removedDate older than 90 days (the maximum county deed lag), AND it does not already exist in the database from Phase 1 (deed_records), assume it is an Expired/Withdrawn listing and drop it.

- Only properties with a removedDate within the last 0–90 days will be given sale_confidence = 'low' and queued for BatchData.
- Older false alarms will be assigned sale_confidence = 'REJECTED' and excluded from the validation queue.
- This rule is applied in processFetchChunk/entry.ts after the merge and deduplication phase.

8.4 API Key Management and Authentication Error Patterns

- HTTP 401 Unauthorized: Missing or invalid X-API-Key header. HALT execution immediately — do not retry. Alert admin.
- HTTP 429 Too Many Requests: Implement exponential backoff as specified in Section 5.2. At 10 reps with cross-reference strategy, RentCast overages reach \$746.40/month — monitor quota consumption closely.
- RentCast Foundation plan: 5,000 calls/month. Cross-reference strategy consumes ~218 calls/area vs. 18 calls/area without cross-reference — a 12x increase that exhausts quota at ~22.9 areas/month.
- Overage rate: \$0.06/call beyond the Foundation plan quota. Negotiate a higher tier before scaling past 3 reps with cross-reference architecture enabled.

8.5 Async Webhook Architecture for BatchData

The Reduce RentCast False Alarms document proposes transitioning BatchData validation from a synchronous 1-by-1 loop to an async bulk job using POST

<https://api.batchdata.com/api/v1/property/lookup/async>. The following open questions must be resolved before implementing the async webhook architecture:

Table 10 — Open Questions Requiring Resolution Before Production Deployment

Open Question	Risk Level	Resolution Required Before
Can Base44 expose a publicly reachable batchDataWebhookCallback function without requiring FirstKnock user authentication?	HIGH	Phase 2 BatchData async implementation
Is 90 days the safe cutoff for all target counties? If a home doesn't appear in county deeds 90 days after going Inactive, is automatic discard acceptable?	MEDIUM	processFetchChunk deployment

Does saleDateRange=0 (lower bound of 0 in min:max syntax) behave as documented or default to 1?	LOW	Any use of saleDateRange=0:45 range syntax in production
What is RentCast's actual listing data refresh cadence? Daily or weekly polling affects lastSeenDate signal reliability.	MEDIUM	Heuristic scoring model calibration

Document prepared by FirstKnock Engineering. For questions about this specification, contact the API integration team. All findings are based on official RentCast API documentation, BatchData API documentation, and internal research reports as of March 31, 2026. Refer to the RentCast API Deep Research Report (Blocks 1 & 2) and the RentCast API Technical Audit for FirstKnock.online for API-specific implementation details not covered in this spec.